Course Overview

Python for AI-Driven Automation and Business Data Science

Course materials



Contents

1. The big picture
2. Structure
3. Learning paths
4. How to study
5. What you'll build
6. Getting started

Contents
1. The big picture
2. Structure
3. Learning paths
4. How to study
5. What you'll build
6. Getting started

Outline

1 The big picture

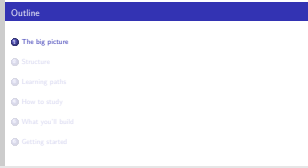
2 Structure

3 Learning paths

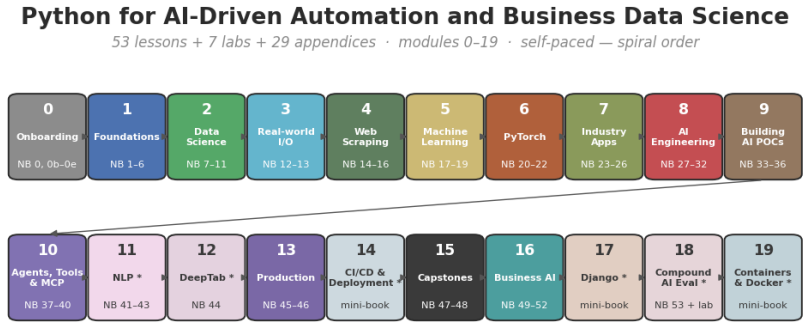
4 How to study

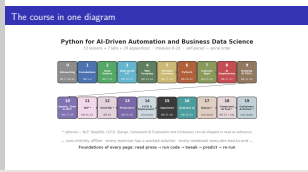
5 What you'll build

6 Getting started



The course in one diagram





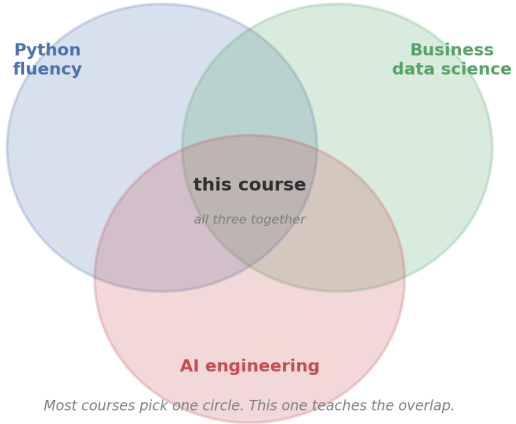
~5 min. Point: show the whole shape once, so everything later has somewhere to attach.

Trace the spine with a finger — Python → data → ML → AI → shipping — and say the one sentence that matters: **each module is a prerequisite for the next, so the order is not arbitrary.**

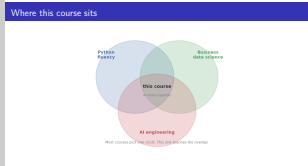
Point out the starred optional modules now to defuse the intimidation: 20 modules is not 20 obligations.

Do not read the boxes aloud. They will read faster than you can speak, and the detail comes three slides later.

Where this course sits



Most courses pick one circle. This one teaches the overlap.



~4 min. Point: this is not a pure programming course, not a pure statistics course, and not a prompt-engineering course — it is the overlap.

The overlap is the employable part: plenty of people can do one circle. Doing all three is what lets you take a business problem and return a working, defensible system.

Ask the room: "Which circle are you strongest in?" Useful for you (calibrates pace) and for them (they find their pairs).

Three skills, taught together

1. Python fluency

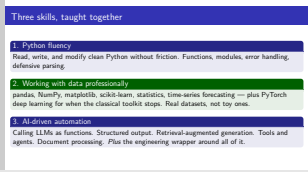
Read, write, and modify clean Python without friction. Functions, modules, error handling, defensive parsing.

2. Working with data professionally

pandas, NumPy, matplotlib, scikit-learn, statistics, time-series forecasting — plus PyTorch deep learning for when the classical toolkit stops. Real datasets, not toy ones.

3. AI-driven automation

Calling LLMs as functions. Structured output. Retrieval-augmented generation. Tools and agents. Document processing. *Plus* the engineering wrapper around all of it.



~**6 min.** Point: name the three skills concretely so "AI course" stops being vague.

The phrase to land is "LLMs as functions." Most students arrive thinking of AI as a chat window. This course treats a model as a component you call from code, with typed inputs and outputs, inside a program you control. That reframe is most of the difference between a demo and a product.

Stress "real datasets, not toy ones" — they'll meet messy data early, on purpose.

"The engineering wrapper" is the honest bit: evaluation, cost, monitoring, deployment. It's the least glamorous and the most decisive.

Outline

1 The big picture

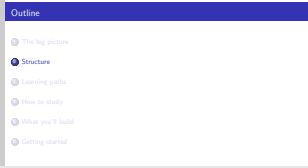
2 Structure

3 Learning paths

4 How to study

5 What you'll build

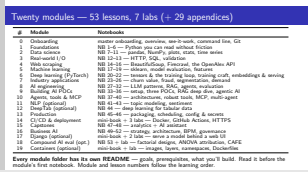
6 Getting started



Twenty modules — 53 lessons, 7 labs (+ 29 appendices)

#	Module	Notebooks
0	Onboarding	master onboarding, overview, see-it-work, command line, Git
1	Foundations	NB 1–6 — Python you can read without friction
2	Data science	NB 7–11 — pandas, NumPy, plots, stats, time series
3	Real-world I/O	NB 12–13 — HTTP, SQL, validation
4	Web scraping	NB 14–16 — BeautifulSoup, Firecrawl, the OpenAlex API
5	Machine learning	NB 17–19 — sklearn, model evaluation, features
6	Deep learning (PyTorch)	NB 20–22 — tensors & the training loop, training craft, embeddings & serving
7	Industry applications	NB 23–26 — churn value, fraud, segmentation, demand
8	AI engineering	NB 27–32 — LLM patterns, RAG, agents, evaluation
9	Building AI POCs	NB 33–36 — setup, three POCs, RAG deep dive, agentic AI
10	Agents, tools & MCP	NB 37–40 — architectures, robust tools, MCP, multi-agent
11	NLP (optional)	NB 41–43 — topic modeling, sentiment
12	DeepTab (optional)	NB 44 — deep learning for tabular data
13	Production	NB 45–46 — packaging, scheduling, config & secrets
14	CI/CD & deployment	mini-book + 3 labs — Docker, GitHub Actions, HTTPS
15	Capstones	NB 47–48 — analytics + AI assistant
16	Business AI	NB 49–52 — strategy, architecture, BPM, governance
17	Django (optional)	mini-book + 2 labs — serve a model behind a web UI
18	Compound AI eval (opt.)	NB 53 + lab — factorial designs, ANOVA attribution, CAFE
19	Containers (optional)	mini-book + lab — images, layers, namespaces, Dockerfiles

Every module folder has its own README — goals, prerequisites, what you'll build. Read it before the module's first notebook. Module and lesson numbers follow the learning order.


Twenty modules — 53 lessons, 7 labs (+ 29 appendices)

#	Module	Notebooks
0	Onboarding	master onboarding, overview, see-it-work, command line, Git
1	Foundations	NB 1–6 — Python you can read without friction
2	Data science	NB 7–11 — pandas, NumPy, plots, stats, time series
3	Real-world I/O	NB 12–13 — HTTP, SQL, validation
4	Web scraping	NB 14–16 — BeautifulSoup, Firecrawl, the OpenAlex API
5	Machine learning	NB 17–19 — sklearn, model evaluation, features
6	Deep learning (PyTorch)	NB 20–22 — tensors & the training loop, training craft, embeddings & serving
7	Industry applications	NB 23–26 — churn value, fraud, segmentation, demand
8	AI engineering	NB 27–32 — LLM patterns, RAG, agents, evaluation
9	Building AI POCs	NB 33–36 — setup, three POCs, RAG deep dive, agentic AI
10	Agents, tools & MCP	NB 37–40 — architectures, robust tools, MCP, multi-agent
11	NLP (optional)	NB 41–43 — topic modeling, sentiment
12	DeepTab (optional)	NB 44 — deep learning for tabular data
13	Production	NB 45–46 — packaging, scheduling, config & secrets
14	CI/CD & deployment	mini-book + 3 labs — Docker, GitHub Actions, HTTPS
15	Capstones	NB 47–48 — analytics + AI assistant
16	Business AI	NB 49–52 — strategy, architecture, BPM, governance
17	Django (optional)	mini-book + 2 labs — serve a model behind a web UI
18	Compound AI eval (opt.)	NB 53 + lab — factorial designs, ANOVA attribution, CAFE
19	Containers (optional)	mini-book + lab — images, layers, namespaces, Dockerfiles

Every module folder has its own README — goals, prerequisites, what you'll build. Read it before the module's first notebook. Module and lesson numbers follow the learning order.

~**8 min.** Point: the full map. Let them *scan* — don't narrate 20 rows.

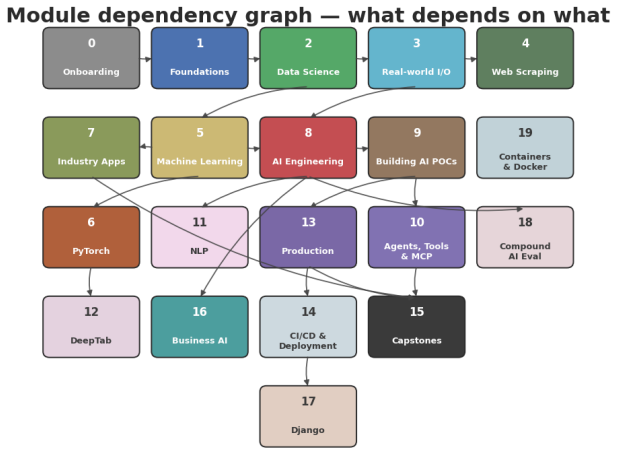
Call out only the shape: modules 1–5 are the skill spine, 7–12 the applications, 13–19 the shipping and judgement half. Then name the optional ones (11, 12, 14, 17, 18, 19) explicitly — that's the anxiety reducer.

The footnote is the actionable instruction: every module has a README with goals and prerequisites. Students who skip them get lost; students who read them never wonder whether they're ready.

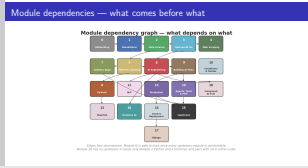
Ask the room: "Anything here you're surprised to see in an AI course?" Usually CI/CD or Django — and the answer is the whole thesis: shipping is the job.

Maintainer note: the counts on this slide and the study slide are hand-maintained. If you've added a module, re-check them against the repo README.

Module dependencies — what comes before what



Edges flow downstream. Module N is safe to start once every upstream module is comfortable.
Module 19 has no upstream: it needs only Module-1 Python and a terminal, and pairs with 14 in either order.



~5 min. Point: it is a graph, not a queue — several modules are safe to take in any order once their upstreams are done.

Use it to answer the question they're all silently asking: "can I skip ahead to the AI parts?" Follow the edges into module 8 — it needs 3 and 5. Skipping means you'll be debugging pandas errors instead of learning AI engineering.

Note module 19 sits as a root: it needs only Module-1 Python and a terminal, so it can be read any time — before or after module 14.

This graph is generated from the repo's module tables, so it stays true after a renumber.

Outline

1 The big picture

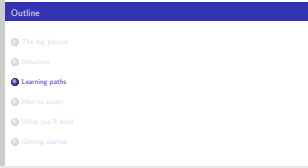
2 Structure

3 Learning paths

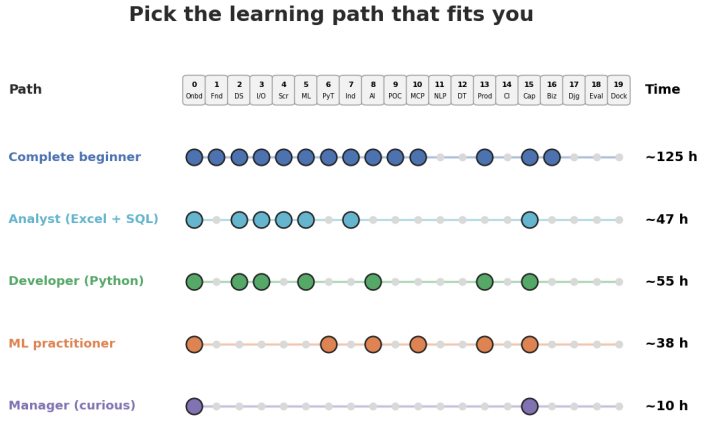
4 How to study

5 What you'll build

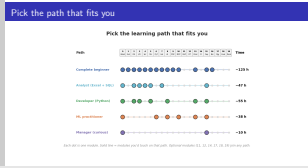
6 Getting started



Pick the path that fits you



Each dot is one module. Solid line = modules you'd touch on that path. Optional modules (11, 12, 14, 17, 18, 19) join any path.



~**4 min.** Point: five paths, wildly different time budgets — 10 h to 125 h. Nobody does all of it unless they want to.

Let them find their row and read across. The greyed dots are modules that path skips, and skipping is *endorsed*, not a compromise.

For a taught cohort, say plainly which path your syllabus follows and which modules are examinable — otherwise this slide creates uncertainty rather than removing it.

Next slide has the detail; keep this one to orientation.

When to pick which path

Complete beginner

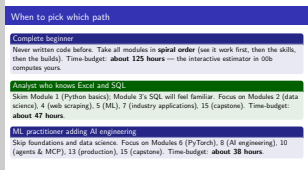
Never written code before. Take all modules in **spiral order** (see it work first, then the skills, then the builds). Time-budget: **about 125 hours** — the interactive estimator in 00b computes yours.

Analyst who knows Excel and SQL

Skim Module 1 (Python basics); Module 3's SQL will feel familiar. Focus on Modules 2 (data science), 4 (web scraping), 5 (ML), 7 (industry applications), 15 (capstone). Time-budget: **about 47 hours**.

ML practitioner adding AI engineering

Skip foundations and data science. Focus on Modules 6 (PyTorch), 8 (AI engineering), 10 (agents & MCP), 13 (production), 15 (capstone). Time-budget: **about 38 hours**.



~8 min. Point: give explicit permission to skip. The most common failure is a capable analyst grinding through Module 1 out of duty and quitting from boredom.

The instruction that matters: **skim, don't skip blind.** Open the module README, check the goals, and if you can already do all of them, move on — and come back the moment something doesn't parse.

The hour figures are honest and worth defending — 125 hours for a true beginner. Underselling it produces dropouts at week three. The estimator in notebook 00b computes a personal number from their answers.

Ask the room: "Which of these three is closest to you?" Hands up per path — tells you instantly what pace the cohort needs.

Twelve weeks, self-paced

A self-paced 12-week schedule

Week 1	Week 2	Week 3	Week 4	Week 5	Week 6
Onboarding + Python I	Python II + Git & Copilot	Data science I	Data science II + real-world I/O	Machine learning	Deep learning (PyTorch)
NB 0–3	*NB 4–6, 51*	*NB 7–9*	*NB 10–13*	*NB 17–19*	*NB 20–22*
~8 h focus	**~10 h focus**	**~8 h focus**	**~10 h focus**	**~9 h focus**	**~9 h focus**

Week 7	Week 8	Week 9	Week 10	Week 11	Week 12
Industry applications	AI engineering I	AI engineering II + production	Business AI + LLM theory	Build the POCs + agents & MCP	Web scraping + capstones
NB 23–26	*NB 27–29*	*NB 30–32, 45–46*	*NB 49–52, 27*	*NB 33–40*	*NB 14–16, 47–48*
~11 h focus	**~9 h focus**	**~12 h focus**	**~9 h focus**	**~15 h focus**	**~12 h focus**

≈ 8–15 hours of focused study per week · more time → faster, less time → split a week in two

Got more time? Move faster. Got less? Split one week into two.



~4 min. Point: a concrete, self-paced 12-week shape — 8–15 h/week. This is the answer to "how long will this actually take?"

Note the weeks are not uniform: week 11 (POCs + agents) is the heaviest at ~15 h. Flag it now so it isn't a surprise.

If you're teaching this as a taught course, say clearly that this is the self-paced plan and your lecture schedule differs — otherwise students try to follow both. The fast track's 12-lecture plan is the taught-course version.

The line to say: consistency beats intensity. Six focused hours a week for twelve weeks beats two heroic weekends.

Outline

1 The big picture

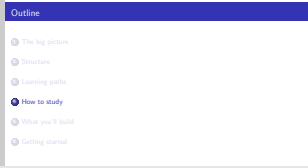
2 Structure

3 Learning paths

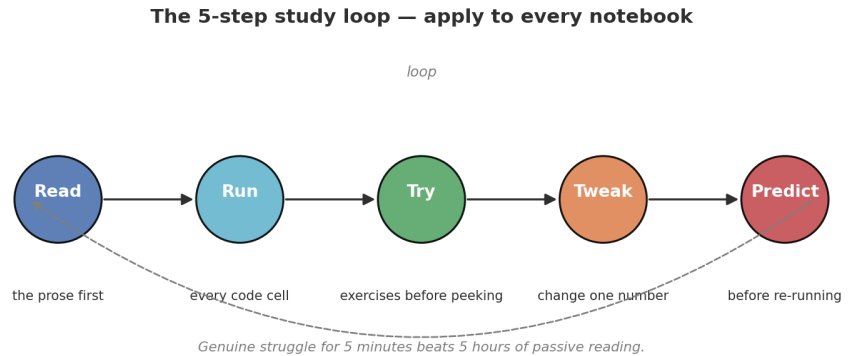
4 How to study

5 What you'll build

6 Getting started



The five-step study loop



Apply this loop to *every* notebook. Genuine struggle beats passive reading.



~6 min — the highest-value slide for outcomes.

Point: *how* to study, not what. Read → Run → Try → Tweak → Predict.

Predict-before-you-run is the step that does the work and the one everyone drops. Saying the expected output out loud before pressing shift-enter converts passive reading into a test with immediate feedback. When the prediction is wrong, that's the learning moment.

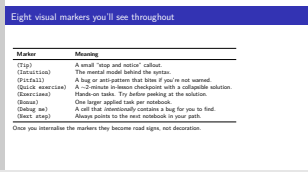
Be blunt about the failure mode: running every cell and watching output scroll past feels productive and teaches almost nothing. Ask them how they'd know the difference the next morning.

Ask the room: "Who has read a whole tutorial and remembered none of it?" Every hand. That's why this slide exists.

Eight visual markers you'll see throughout

Marker	Meaning
`(Tip)`	A small "stop and notice" callout.
`(Intuition)`	The mental model behind the syntax.
`(Pitfall)`	A bug or anti-pattern that bites if you're not warned.
`(Quick exercise)`	A ∼2-minute in-lesson checkpoint with a collapsible solution.
`(Exercises)`	Hands-on tasks. Try *before* peeking at the solution.
`(Bonus)`	One larger applied task per notebook.
`(Debug me)`	A cell that *intentionally* contains a bug for you to find.
`(Next step)`	Always points to the next notebook in your path.

Once you internalise the markers they become road signs, not decoration.


Eight visual markers you'll see throughout

Marker	Meaning
(Tip)	A small "stop and notice" callout.
(Intuition)	The mental model behind the syntax.
(Pitfall)	A bug or anti-pattern that bites if you're not warned.
(Quick exercise)	A ∼2-minute in-lesson checkpoint with a collapsible solution.
(Exercises)	Hands-on tasks. Try before peeking at the solution.
(Bonus)	One larger applied task per notebook.
(Debug me)	A cell that intentionally contains a bug for you to find.
(Next step)	Always points to the next notebook in your path.

Once you internalise the markers they become road signs, not decoration.

∼5 min. Point: the notebooks have a consistent visual grammar — learn it once, navigate 134 notebooks.

The two to dwell on: **Quick exercise** (a ∼2-minute checkpoint with a collapsible solution — the rhythm of every lesson) and **Debug me** (a cell that is *intentionally* broken).

Warn them explicitly about Debug me, or someone will spend twenty minutes assuming the course has a typo. The bug is the exercise.

Tell them to resist opening solutions early — reading a solution feels like understanding and isn't. The struggle is where the learning is.

Checkpoints, quizzes, and the fast track

389 in-lesson checkpoints — the rhythm of every lesson

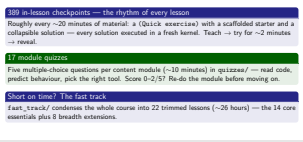
Roughly every ∼20 minutes of material: a (`Quick exercise`) with a scaffolded starter and a collapsible solution — every solution executed in a fresh kernel. Teach → try for ∼2 minutes → reveal.

17 module quizzes

Five multiple-choice questions per content module (∼10 minutes) in `quizzes/` — read code, predict behaviour, pick the right tool. Score 0–2/5? Re-do the module before moving on.

Short on time? The fast track

`fast_track/` condenses the whole course into 22 trimmed lessons (∼26 hours) — the 14 core essentials plus 8 breadth extensions.



∼6 min. Point: three feedback mechanisms at three timescales — checkpoints (minutes), quizzes (per module), fast track (an alternative route).

If you're teaching live, this slide is about you as much as them: the 389 checkpoints are the built-in pause points. Teach ∼20 min, set the checkpoint, give them ∼2 min, reveal. That's the lecture rhythm, and it needs no preparation.

The quiz rule is worth stating plainly: score 0–2 out of 5 and redo the module before moving on. The dependency graph means weak foundations compound.

Mention the fast track exists but don't oversell it to a taught cohort — it's for people with a hard time budget, and it trims depth to get there.

Outline

1 The big picture

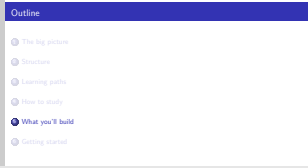
2 Structure

3 Learning paths

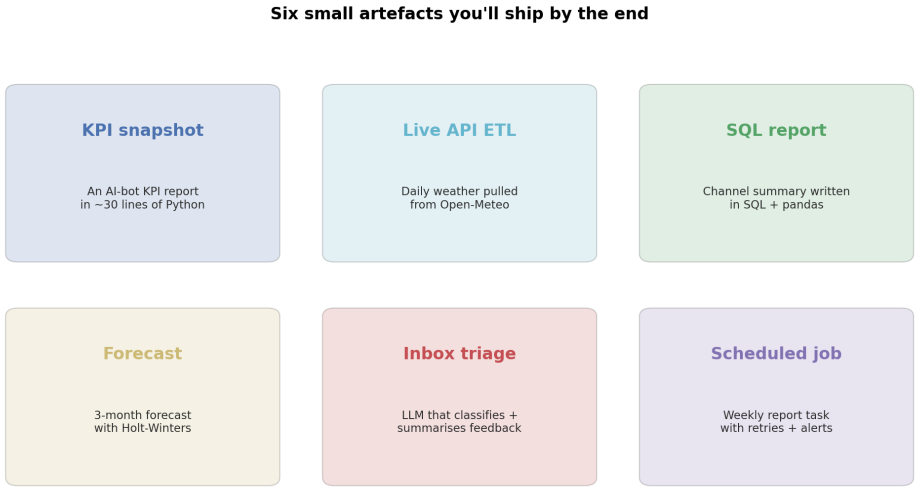
4 How to study

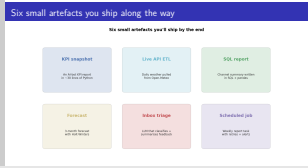
5 What you'll build

6 Getting started



Six small artefacts you ship along the way





~5 min. Point: they finish with things that exist, not just notes. Motivation slide — keep the energy up.

The framing that lands with career-minded students: these are portfolio pieces you can show and *explain*. Being able to defend a design decision in an interview is what separates them from someone who followed a tutorial.

Keep it brief — the detail is in the modules, and the capstones are on the next slide.

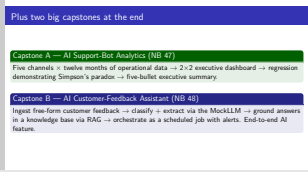
Plus two big capstones at the end

Capstone A — AI Support-Bot Analytics (NB 47)

Five channels × twelve months of operational data → 2×2 executive dashboard → regression demonstrating Simpson's paradox → five-bullet executive summary.

Capstone B — AI Customer-Feedback Assistant (NB 48)

Ingest free-form customer feedback → classify + extract via the MockLLM → ground answers in a knowledge base via RAG → orchestrate as a scheduled job with alerts. End-to-end AI feature.



~6 min. Point: two capstones, deliberately different — A is the analytics story, B is the AI system.

Capstone A's real lesson is Simpson's paradox — the aggregate trend reverses when you segment. It's the single most useful statistical trap for anyone presenting numbers to management, and it lands far harder in a project than on a slide.

Capstone B is the whole AI-engineering stack in one artefact: classify, extract, ground with RAG, schedule, alert. If they can explain that pipeline, they can hold a technical conversation about production AI.

Both end in a *decision or a summary a manager could act on* — that's the course's signature, and it's what makes them portfolio-worthy.

Outline

1 The big picture

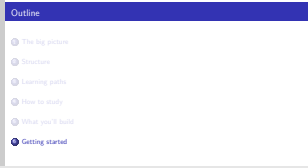
2 Structure

3 Learning paths

4 How to study

5 What you'll build

6 Getting started



LLM providers — local and hosted, four options

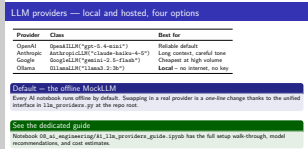
Provider	Class	Best for
OpenAI	`OpenAILLM("gpt-5.4-mini")`	Reliable default
Anthropic	`AnthropicLLM("claude-haiku-4-5")`	Long context, careful tone
Google	`GoogleLLM("gemini-2.5-flash")`	Cheapest at high volume
Ollama	`OllamaLLM("llama3.2:3b")`	**Local** – no internet, no key

Default = the offline MockLLM

Every AI notebook runs offline by default. Swapping in a real provider is a *one-line* change thanks to the unified interface in `llm_providers.py` at the repo root.

See the dedicated guide

Notebook `08_ai_engineering/A1_llm_providers_guide.ipynb` has the full setup walk-through, model recommendations, and cost estimates.



~**6 min.** Point: the course is **100 % offline by default**. Say this early and clearly — it removes the biggest barrier and the biggest anxiety.

Nobody needs an API key, a credit card, or an account to complete this course. Every AI notebook runs against a built-in MockLLM. Repeat it; students who half-heard it will worry for weeks.

Swapping in a real provider is a one-line change through the shared interface in `llm_providers.py` — that's a service-oriented boundary, and NB 50 names it as one.

Maintainer note: the model names here mirror the defaults in `llm_providers.py`. If you update one, update the other — and expect this table to date quickly.

What you need to install

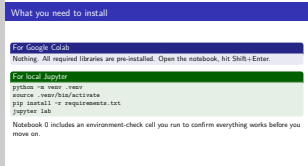
For Google Colab

Nothing. All required libraries are pre-installed. Open the notebook, hit Shift+Enter.

For local Jupyter

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
jupyter lab
```

Notebook 0 includes an environment-check cell you run to confirm everything works before you move on.



~8 min — and budget more if this is session one with a fresh cohort.

Point: two routes in. **Colab needs nothing** — that is the answer for anyone whose laptop is fighting them, and it takes setup off the critical path entirely.

If you're teaching live, do the local setup with the room now, not later. Environment problems are the number-one reason people quietly stop, and they're much cheaper to fix with a neighbour than alone at home. Expect the usual: wrong Python version, activation not persisting, corporate proxies.

Point everyone at the environment-check cell in notebook 0 — it's the definitive answer to "is my setup right?"

Have the Colab link ready as the escape hatch for anyone still stuck after ten minutes; move on and debug in the break.

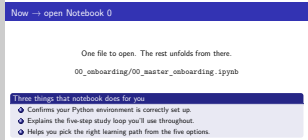
Now → open Notebook 0

One file to open. The rest unfolds from there.

`00_onboarding/00_master_onboarding.ipynb`

Three things that notebook does for you

1. Confirms your Python environment is correctly set up.
2. Explains the five-step study loop you'll use throughout.
3. Helps you pick the right learning path from the five options.



~4 min. Point: reduce the whole deck to one action — open one file.

The most common way to fail at this course is never starting because 134 notebooks feels like too much. One file. The rest unfolds.

If you have five minutes, open it on screen now and let them watch the environment check run. Seeing it work is worth more than describing it.

If your cohort is following a fixed syllabus, say so here so nobody spends an evening on the path-picker.

One last reminder before you start

Five-step study loop, in your pocket

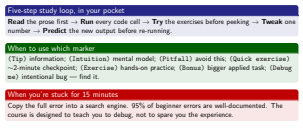
Read the prose first → **Run** every code cell → **Try** the exercises before peeking → **Tweak** one number → **Predict** the new output before re-running.

When to use which marker

`(Tip)` information; `(Intuition)` mental model; `(Pitfall)` avoid this; `(Quick exercise)` ∼2-minute checkpoint; `(Exercise)` hands-on practice; `(Bonus)` bigger applied task; `(Debug me)` intentional bug — find it.

When you're stuck for 15 minutes

Copy the full error into a search engine. 95% of beginner errors are well-documented. The course is designed to teach you to debug, not to spare you the experience.



∼5 min. Point: the debugging attitude. This is the slide that keeps people from quitting in week two.

Normalise the errors: a red traceback is not a sign they're unsuited to this. Professionals see them all day; the difference is they read them instead of panicking. Tell them to read the *last* line first.

The 15-minute rule is a real technique — struggle is productive up to a point, then it's just attrition. Search the exact error text, then ask a person.

Nice callback: NB 27 explains *why* the assistant will confidently invent a fix; NB 51 gives the review checklist. The course teaches supervision, not avoidance.

If you're teaching live, this is where you name your office hours or forum. Say it concretely.

Welcome.

134 notebooks await you.

Open Module 0 next.

Welcome.
134 notebooks await you.
Open Module 0 next.

∼2 min — close warmly and stop.

Point: end on the invitation, not on logistics. Resist adding one more announcement here.

If you want one closing line, make it the promise the course actually keeps: *by the end, you will have shipped something that runs, and you'll be able to explain every part of it.*

Then give the single next action out loud — open `00_master_onboarding.ipynb` before the next session — and take questions.